EE337 - Microprocessors Lab
I2C Device Driver Project

Sai Vishal Reddy Malireddy, 24B1293
Pratyush Kumar Masanta, 24B1294

April 17, 2026

1 Objective

The objective of this project is to develop a basic device driver for I2C communication on the PT-51 board. The system interfaces with external peripherals such as the BMP280 pressure sensor and DS1307 RTC module. The project progresses from low-level register manipulation to abstraction via reusable driver functions.

2 Background

I2C (Inter-Integrated Circuit) is a two-wire communication protocol consisting of:

- SDA (Serial Data Line)
- SCL (Serial Clock Line)

It supports multiple devices on the same bus using unique addresses. Communication is initiated by a master device and involves START, STOP, ACK, and data transfer sequences.

According to the AT89C5131A datasheet, the I2C interface (also called the TWI - Two Wire Interface) is controlled using registers such as SSCON, SSDAT, and SSCS which manage control, data transfer, and status respectively.

3 Phase 1: BMP280 Identification Register Read

In this phase, a bare-metal implementation of I2C was developed to read the **WHO_AMI register (0xD0)** of the BMP280 sensor.

The BMP280's slave address is 0x76 if it's SDO pin is LOW and 0x77 if it is HIGH.

The I2C bus uses open-drain configuration requiring external pull-up resistors (typically 10 kΩ) to ensure proper logic HIGH levels, as per standard I2C design guidelines.

3.1 Approach

- Generate START condition
- Send slave address (0x76) with write bit
- Send register address (0xD0)
- Generate repeated START
- Send slave address with read bit
- Read data and send NACK
- Generate STOP condition

The BMP280 datasheet specifies that register 0xD0 contains the chip ID.

3.2 Key Code Snippet

```c
unsigned char BMP280_ReadID() {
    unsigned char id;

    I2C_Start(0x08);

    I2C_Write((BMP280_ADDR << 1) | 0, 0x18);
    I2C_Write(0xD0, 0x28);

    I2C_Start(0x10);

    I2C_Write((BMP280_ADDR << 1) | 1, 0x40);

    id = I2C_Read(0, 0x58);

    I2C_Stop();

    return id;
}
```

3.3 I2C Communication Flow

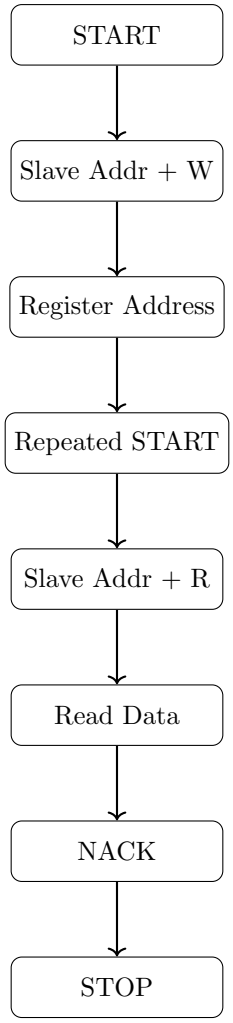


Figure 1: I2C Read Sequence for BMP280 Register Access

3.4 I2C Timing Diagram

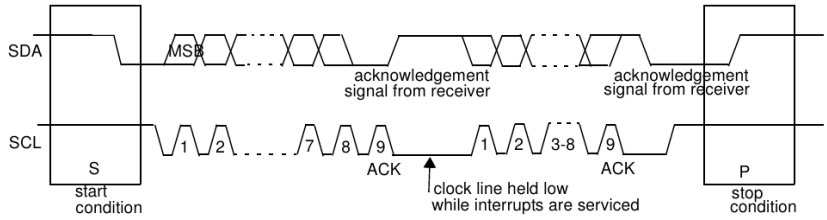


Figure 2: Timing Waveform

The timing diagram illustrates the fundamental I2C operations. The START condition is generated when SDA goes LOW while SCL is HIGH. Data bits are transmitted synchronously with the clock, and the receiver acknowledges each byte by pulling SDA LOW during the ninth clock pulse. The STOP condition is generated when SDA transitions from LOW to HIGH while SCL is HIGH.

3.5 SSCS Status Codes for I2C Master Mode

Status Code	Description
0x08	START condition transmitted
0x10	Repeated START transmitted
0x18	SLA+W transmitted, ACK received
0x20	SLA+W transmitted, NACK received
0x28	Data byte transmitted, ACK received
0x30	Data byte transmitted, NACK received
0x38	Arbitration lost
0x40	SLA+R transmitted, ACK received
0x48	SLA+R transmitted, NACK received
0x50	Data byte received, ACK returned
0x58	Data byte received, NACK returned

Table 1: SSCS Status Codes for Master Transmitter and Receiver Modes

These status codes are stored in the SSCS register and indicate the current state of the I2C hardware. In this phase, the expected status codes were checked after each operation to ensure correct communication with the slave device. Any deviation from expected values was treated as an error condition and handled appropriately.

3.6 Result

The chip ID (**0x58**) was successfully read and displayed on the LCD, confirming correct I2C communication.

4 Phase 2: I2C Device Driver Library

In this phase, a reusable I2C driver library was Developed.

4.1 Driver Functions

The following functions were created:

- I2C_Start()
- I2C_Stop()
- I2C_Write()
- I2C_Read()
- I2C_Ack()
- I2C_Nack()

Additionally, higher-level functions were also implemented:

- I2C_ReadByte(device_addr, reg_addr)
- I2C_WriteByte(device_addr, reg_addr, data)

4.2 Key Code Snippet

```
static void I2C_Wait() {
    while(!(SSCON & SI));   // wait for completion
}

void I2C_Start() {
    SSCON |= STA;
    SSCON &= ~SI;

    I2C_Wait();
    SSCON &= ~STA;
}

void I2C_Stop() {
    SSCON |= STO;
    SSCON &= ~SI;

    while(SSCON & STO); //since the STOP condition is
        automatically cleared
}

void I2C_Write(unsigned char write_data) {
    SSDAT = write_data;
```

```
    SSCON &= ~SI;

    I2C_Wait();
}

unsigned char I2C_Read() {
    SSCON &= ~SI;

    I2C_Wait();

    return SSDAT;
}

void I2C_Ack() {
    SSCON |= AA;
}

void I2C_Nack() {
    SSCON &= ~AA;
}

unsigned char I2C_ReadByte(unsigned char device_addr,
    unsigned char reg_addr) { //for reading just one byte
    from the device
    unsigned char read_data;

    I2C_Start();

    I2C_Write((device_addr << 1) | 0); // Write mode
    I2C_Write(reg_addr);

    I2C_Start(); // Repeated start

    I2C_Write((device_addr << 1) | 1); // Read mode

    I2C_Nack(); // NACK before reading the byte
    read_data = I2C_Read();

    I2C_Stop();

    return read_data;
}

void I2C_WriteByte(unsigned char device_addr, unsigned char
    reg_addr, unsigned char write_data) { //for writing just
    one byte to the device
    I2C_Start();

    I2C_Write((device_addr << 1) | 0); // Write mode
    I2C_Write(reg_addr);
```

```
    I2C_Write(write_data);

    I2C_Stop();
}
```

4.3 Result

This abstraction allows application-level code to interact with devices without handling low-level register operations directly (Kind of like an API).

5 Phase 3: RTC and Temperature Integration

In this phase, the developed I2C driver was used to interface:

- DS1307 RTC (Time)
- BMP280 (Temperature sensing)

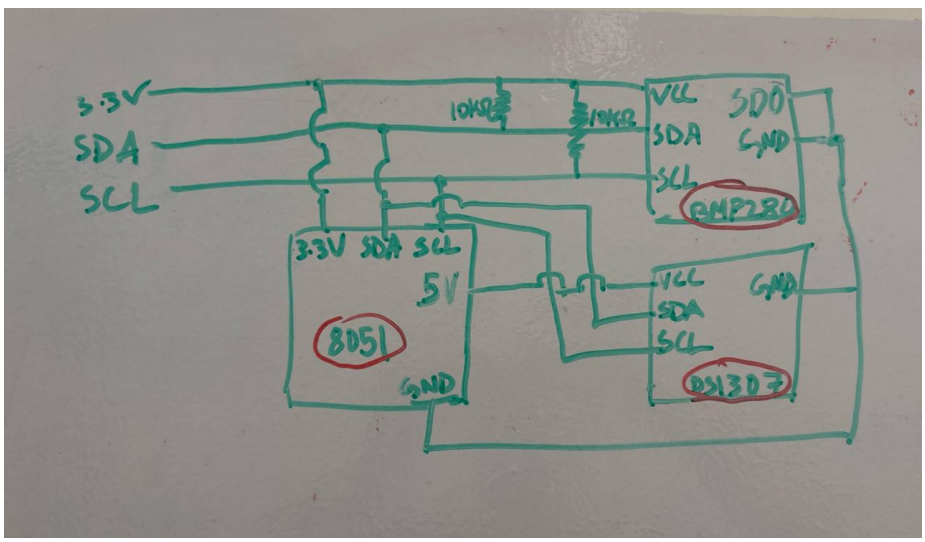


Figure 3: Main Circuit

5.1 RTC Interfacing

The DS1307 provides time in BCD format, which was converted to decimal for display.

```
int bcd_to_dec(unsigned char val)
{
    return ((val >> 4) * 10) + (val & 0x0F);
```

```
}
```

Time registers (seconds, minutes, hours) were read using sequential I2C transactions.

Input time would be taken from the UART terminal and the RTC would initialize from that time and the real time is displayed on the LCD.

5.2 Temperature Calculation

Raw temperature data from BMP280 was converted using calibration parameters:

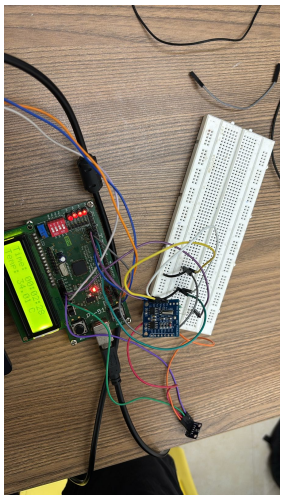
```
int get_temp_in_C(long raw_T)
{
    long var1, var2, t_fine;
    int T;

    var1 = ((((raw_T >> 3) - ((long)dig_T1 << 1))) * ((long)
        dig_T2)) >> 11;

    var2 = (((((raw_T >> 4) - ((long)dig_T1)) *
              ((raw_T >> 4) - ((long)dig_T1))) >> 12) *
            ((long)dig_T3)) >> 14;

    t_fine = var1 + var2;

    T = (t_fine * 5 + 128) >> 8;

    return T;
}
```

The raw temperature was taken from the BMP280 sensor in a 20-bit format (MSB(8) + LSB(8) + XLSB(4)) using the I2C Read sequence.

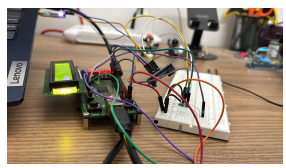
The final temperature in $^{\circ}C$ was displayed on the second line of the LCD.

6 Observations and Results

- Successful I2C communication with BMP280 and DS1307
- Correct chip ID read from BMP280
- RTC time correctly set and displayed
- Temperature values computed accurately
- UART communication functioning at 1200 baud
- The RTC time and Temperature values are displayed on both the LCD and the UART terminal, and the readings were programmed to be taken for every one second!

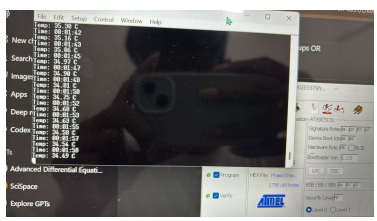


(a) Hardware



(b) Hardware

(c) LCD Output



(d) Teraterm Output

Figure 4: Results

The system demonstrated reliable master-slave communication over I2C.

7 Conclusion

This project successfully demonstrated the development of a basic I2C device driver on an 8051 microcontroller. Starting from low-level register manipulation, the implementation progressed to modular driver design and finally to real-world applications involving RTC and sensor interfacing.

The abstraction of I2C operations into reusable functions reflects the core principles of firmware and device driver development.